END-TO-END RAG SYSTEM REPORT

1. Task Overview

This project builds a retrieval augmented generation (RAG) system for factual question answering. The system receives a question, retrieves relevant passages from a public knowledge resource, and returns a concise answer. The final submitted output is ‘system_outputs/system_output_1.txt‘, with one generated answer per test question.

2. Data Creation

Knowledge Resource

The knowledge resource was collected from public VnExpress RSS feeds and article pages. I used three feeds to cover several factual domains:

- ‘giao_duc‘: education and university-related news.
- ‘khoa_hoc‘: science and research news.
- ‘so_hoa‘: digital technology and product news.

The final raw corpus contains 90 public articles and 76,213 words. Each article is stored with structured metadata and body text:

- source
- category
- domain
- URL
- publication date
- title
- summary
- article body

The raw data is stored in ‘data/raw/news/corpus_long.txt‘. The crawled article records are stored in ‘data/news/articles.jsonl‘, and summary statistics are stored in ‘data/news/metadata.json‘.

Extraction And Cleaning

The data collection script is ‘scripts/prepare_news_rag_data.py‘. It reads RSS feeds, fetches public article pages, extracts title, date, summary, and article paragraphs with BeautifulSoup, and removes repeated whitespace. The cleaned text is converted into a plain text corpus. ‘scripts/build_corpus.py‘ then chunks the corpus into overlapping word windows and writes ‘data/processed/corpus.jsonl‘.

QA Annotation

The QA data was generated from article metadata and body content. The question templates use natural article-title references rather than artificial IDs. Example question types include:

- Source/category/domain questions.
- URL and publication date questions.
- Summary questions.

- Opening, second, third, ending, and first-words body questions.

The final annotated data has 1,000 QA pairs:

- 850 train/development examples in ‘data/train/‘.
- 150 test examples in ‘data/test/‘.

The answer strings are copied directly from the source article fields or article body. This makes each reference answer grounded in the public knowledge resource.

Data Quality

I validated the generated QA files by checking that every question has a corresponding answer line and that the source corpus contains the articles referenced by the question titles. During development, I found that a 50,000-word truncation removed some later articles while the QA generator still produced questions from all 90 articles. I fixed this by keeping the full 90-article corpus, then rebuilt the processed corpus, retrieval index, and system outputs.

This was a single-person project, so a formal two-annotator inter-annotator agreement score was not available. Instead, I performed deterministic consistency checks and manual audits of sampled QA pairs. The main remaining risk is that automatically generated questions are less diverse than fully human-written questions.

3. System Design

Preprocessing

‘scripts/build_corpus.py‘ reads ‘.txt‘, ‘.html‘, and ‘.pdf‘ documents, normalizes whitespace, and chunks text into 900-word windows with 150-word overlap. For the final run, it produced 102 chunks from the news corpus.

Retrieval

The retriever uses ‘sentence-transformers/paraphrase-multilingual-mpnet-base-v2‘ to encode the corpus chunks and questions. Retrieval is based on cosine similarity. Because the QA dataset asks questions by article title, I added an exact quoted-title boost during retrieval. If a question contains a quoted title and a chunk contains that title, the chunk receives an additional score. This improves grounding and avoids retrieving a semantically similar but wrong article.

The optional reranker uses ‘cross-encoder/mmarco-mMiniLMv2-L12-H384-v1‘. It is disabled in the final command with ‘--no-reranker‘ to keep the run practical on the local CPU environment.

Answering

The main system is implemented in ‘src/rag_system/qa.py‘. It uses two answer paths:

1. A fast article-field extractor for the generated title-based QA patterns. It reads fields such as source, category, URL, date, summary, and relevant body sentences from the retrieved article text.
2. A fallback extractive QA reader using ‘letrunglinh/qa_pnc‘ for questions that do not

match the structured patterns.

This keeps the final system grounded in the retrieved documents while still supporting extractive answering for less structured questions.

I also added a Windows-specific workaround in ‘src/rag_system/model_cache.py‘ because PyTorch import was hanging when Windows WMI platform queries stalled. The workaround patches the platform probes before loading Hugging Face libraries.

4. Experiments

I used exact match, token F1, and answer recall, matching the assignment metrics. The evaluation script is ‘scripts/evaluate.py‘.

Development Runs

System variant	Corpus	Notes	Exact match	F1	Answer recall
Early dense retrieval + QA	truncated 50,000-word corpus	Some referenced articles were missing	0.3600	0.4719	0.3600
Title-aware retrieval + structured extractor	truncated 50,000-word corpus	Better retrieval, but still missing later articles	0.6733	0.7207	0.6800
Final system	full 90-article corpus	Full corpus, title-aware retrieval, structured extractor + QA fallback	0.9067	0.9332	0.9133

Final Run Commands

‘‘‘bash
python scripts/build_corpus.py --raw-dir data/raw/news --out data/processed/corpus.jsonl
python scripts/build_index.py --corpus data/processed/corpus.jsonl --out data/processed/index.pkl
python scripts/run_rag.py --questions data/test/questions.txt --index data/processed/index.pkl --out system_outputs/system_output_1.txt --no-reranker
python scripts/evaluate.py --predictions system_outputs/system_output_1.txt --references data/test/reference_answers.txt
‘‘‘

Final local evaluation:

‘‘‘text
exact_match: 0.9067
f1: 0.9332
answer_recall: 0.9133
‘‘‘

5. Analysis

The final system performs well on metadata questions because the answer fields are explicitly present in the retrieved article text. It also performs well on opening and summary questions when the retriever returns the correct title-matched chunk.

Most remaining errors come from body-sentence questions. Long article chunks sometimes include several articles, so extracting the "ending" or "third information" can select a

nearby sentence from the wrong segment if article boundaries are not perfectly isolated. The title boost reduces this problem, but a stronger article-level document store would likely improve these cases further.

The largest improvement came from fixing corpus coverage. The earlier 50,000-word corpus truncation made the test set inconsistent with the knowledge resource because some later articles were absent. Keeping the full 90-article corpus improved both retrieval and extractive answering.

Compared with a closed-book model, the RAG system is more controllable because answers are drawn from collected public documents. It also avoids relying on model memory for publication dates, URLs, source names, and article summaries.

6. Limitations

- The QA set is template-generated, so it is less linguistically diverse than fully human-written questions.
- Formal inter-annotator agreement was not computed because this was completed as a single-person project.
- The final system is optimized for article-title questions; unseen questions without exact article titles may require stronger semantic retrieval and reranking.
- The generated binary retrieval index is not committed to git, so it must be rebuilt with ‘scripts/build_index.py‘.

7. References

- Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- Rajpurkar et al. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text.
- Hugging Face Transformers documentation.
- Sentence Transformers documentation.
- VnExpress public RSS feeds and article pages.